Introduction to Parsers

- Role of Parsers in NLP
- Words and Word Groups (Constituency)
- Types of Parsers in NLP
- Parsing Ambiguity

Introduction

- A parser in natural language processing (NLP) is a critical component that **analyzes the grammatical structure of a sentence.**
- It arranges words into **specific groups such as nouns, verbs, and phrases**.
- It breaks down a sentence into its constituent parts, such as nouns, verbs, and phrases, **to understand the syntactic relationships between them.**
- Parsing is essential for various NLP tasks, including **machine translation, information extraction, and question answering.**
- **POS (Part of Speech Tagging) removes lexical ambiguity** while **Parser removes syntax ambiguity**.

Syntax:

- It refers to the way in which **words are arranged together to form sentences**.
- It involves rules that govern the structure of sentences and **how words are combine to convey meaning.**

Relationship Between Words and Word Groups:

- Parsing finds out the relationships between various **words and word groups (Constituency)** within a sentence.
- It determines how words depend on each other, identifying grammatical functions such as **subject-verb-object relationships and modifiers**.

Examples of Words and Word Groups

Words: Individual words in a sentence, each playing a specific grammatical role.

Nouns: dog, cat, grasshopper, frog, bug

Verbs: chases, avoids, befriends, finds, hunts

Adjectives: quick, green, small

Adverbs: quickly, slowly

Pronouns: he, she, it

Prepositions: in, on, at

Conjunctions: and, but, or

Word Groups (Constituency)

Word Groups (Constituency):Clusters of words that function together as a single unit within a sentence:

Noun Phrases (NP):

The quick brown fox (NP), A small bug (NP), The green grasshopper (NP), The book (NP)

Verb Phrases (VP):

chases the cat (VP), befriends the frog (VP), finds the bug (VP)

Prepositional Phrases (PP):

in the garden (PP), on the leaf (PP), at the pond (PP)

Adjective Phrases (AdjP):

very quick (AdjP), extremely small (AdjP), quite green (AdjP)

Adverb Phrases (AdvP):

very quickly (AdvP), rather slowly (AdvP)

Conjunction Phrases (ConjP):

and then (ConjP), but also (ConjP)

Types of Parsers in NLP

Dependency Parsers: Focus on the **relationships between words**, creating a tree structure where **each node represents a word, and edges represent syntactic dependencies**.

Constituency Parsers: Emphasize the **hierarchical structure of sentences, breaking them down into nested constituents (phrases) and representing this structure in a parse tree.**

Top-Down Parsers: Starts from the **highest-level rule and break it down into its components.**

Bottom-up parsers: Begin with the input and gradually **construct the parse tree by combining components into higher-level structures.**

Dependency Parsers: Dependency parsers analyze relationships between words.

Example: "The cat chases the mouse":

Relationship: **chases (cat, mouse)**

Constituency Parsers: Constituency parsers break sentences into nested phrases:

Example: "**The cat chases the mouse**":

Parse Tree:

(S (NP (**DT The**) (**NN cat**)) (VP (**VBZ chases**) (NP (**DT the**) (**NN mouse**))))

VBZ : Verb, present tense, third person singular.

Top-Down Parsing: Top-down parsing starts from the highest-level rule:

Example: For "The cat chases the mouse":

Starting with S, recursively break down into NP and VP:

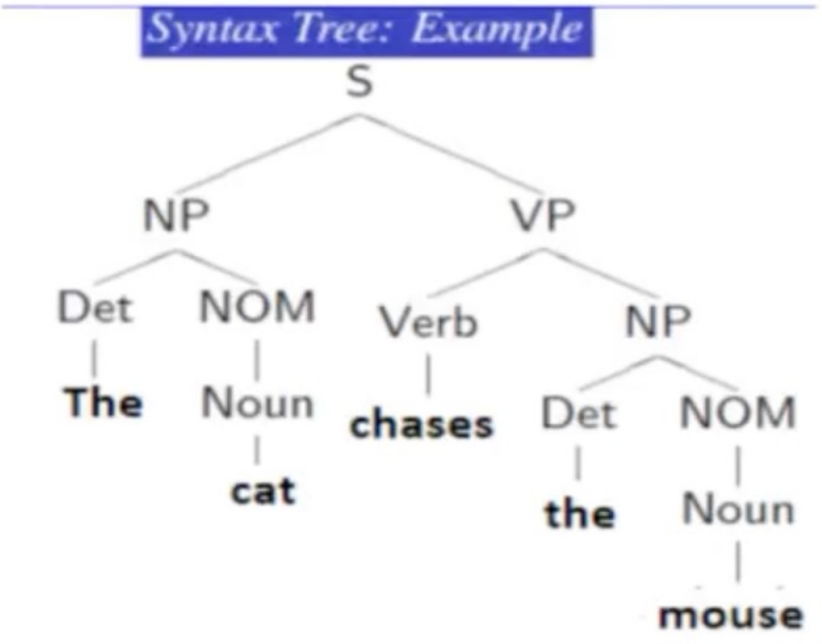
S (NP VP), NP (DT NN), VP (VBZ NP), NP (DT NN)

Bottom-Up Parsing: Bottom-up parsing builds up from input tokens:

Example: Using CYK Algorithm for "The cat chases the mouse":

Combine tokens into NP and VP, then into S:

NP (The cat), VP (chases the mouse), S (NP VP)



Parsing Ambiguity

- Parsing ambiguity refers to the phenomenon where a **single sentence can have multiple valid parse trees, each representing a different syntactic interpretation.**
- This is especially common in natural language processing (NLP) due to the **inherent complexity and flexibility of human language**.
- As sentences grow in length and complexity, the number of possible parses can increase dramatically, demonstrating an "explosive" growth in ambiguity.

Parsing Ambiguity

Ambiguity is Explosive

- I saw the man with the telescope. **2 parses**
- I saw the man on the hill with the telescope. **5 parses**
- I saw the man on the hill in Texas with the telescope. **14 parses**
- I saw the man on the hill in Texas with the telescope at noon. **42 parses**
- I saw the man on the hill in Texas with the telescope at noon on Monday. **132 parses**

These numbers is called **Catalan number**.

Parsing Ambiguity

Ambiguity is Explosive

- I saw the man with the telescope. **2 parses**
- I saw the man on the hill with the telescope. **5 parses**
- I saw the man on the hill in Texas with the telescope. **14 parses**
- I saw the man on the hill in Texas with the telescope at noon. **42 parses**
- I saw the man on the hill in Texas with the telescope at noon on Monday. **132 parses**

These numbers is called **Catalan number**.

I saw the man on the hill with the telescope: 5 Passes

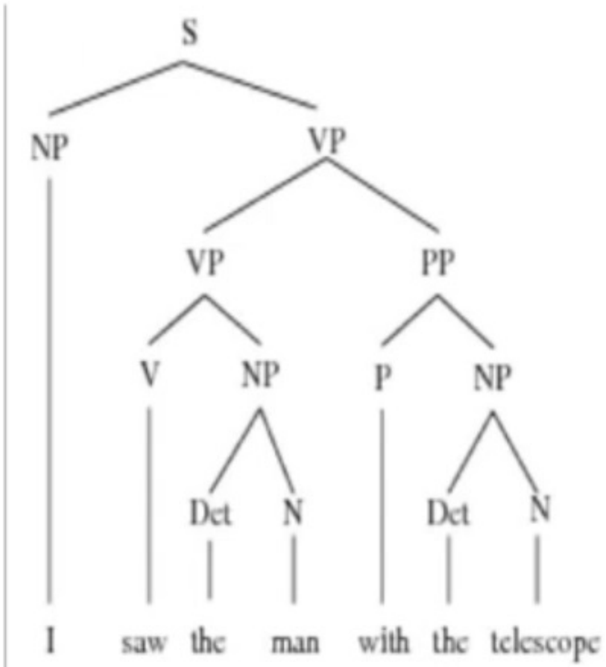
1) I saw the man. The man was on the hill. I was using a telescope.

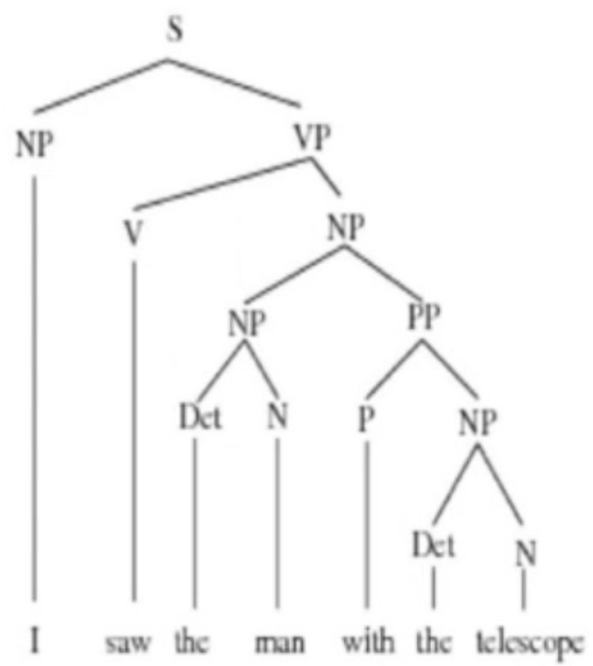
2) There is a man on a hill, whom I am watching, and he has a telescope.

3) I saw the man. The man was on the hill. The hill had a telescope.

4) I saw the man. I was on the hill. I was using a telescope.

5) I saw the man. I was on the hill. The hill had a telescope





- The exponential increase in the number of possible parses as sentences become more complex highlights the challenge of syntactic ambiguity in NLP.
- Parsers must efficiently manage this ambiguity to generate the correct parse tree for accurate language understanding.
- Advanced parsing techniques and algorithms, such as probabilistic context-free grammars (PCFGs) and machine learning-based parsers, are often employed to handle this complexity by assigning probabilities to different parse trees and selecting the most likely one.

Introduction to Parsers

- Modelling Constituency
- Context Free Grammar (CFG)
- Chomsky Normal Form (CNF)
- Top down & Bottom Up Approach

Modelling Constituency

- Modeling constituency in NLP parsing involves **representing the hierarchical structure of sentences by identifying and labeling their constituent parts, such as noun phrases (NP) and verb phrases (VP).**
- This process aims to **break down sentences into nested phrases and construct parse trees that depict their syntactic relationships.**
- Constituency parsing is essential for various NLP tasks, including syntactic analysis, information extraction, and machine translation.
- It enables machines **to understand the grammatical structure of sentences, facilitating accurate interpretation and generation of natural language.**
- Constituency parsers use parsing algorithms like **top-down, bottom-up, or statistical methods to generate parse trees.**
- It also aids in **semantic analysis and downstream applications like named entity recognition and syntactic ambiguity resolution**

- How to arrange words in certain groups?
- How to find automatically word arrangements when new sentences is given?

e.g. In English, I play cricket. **Correct statement**

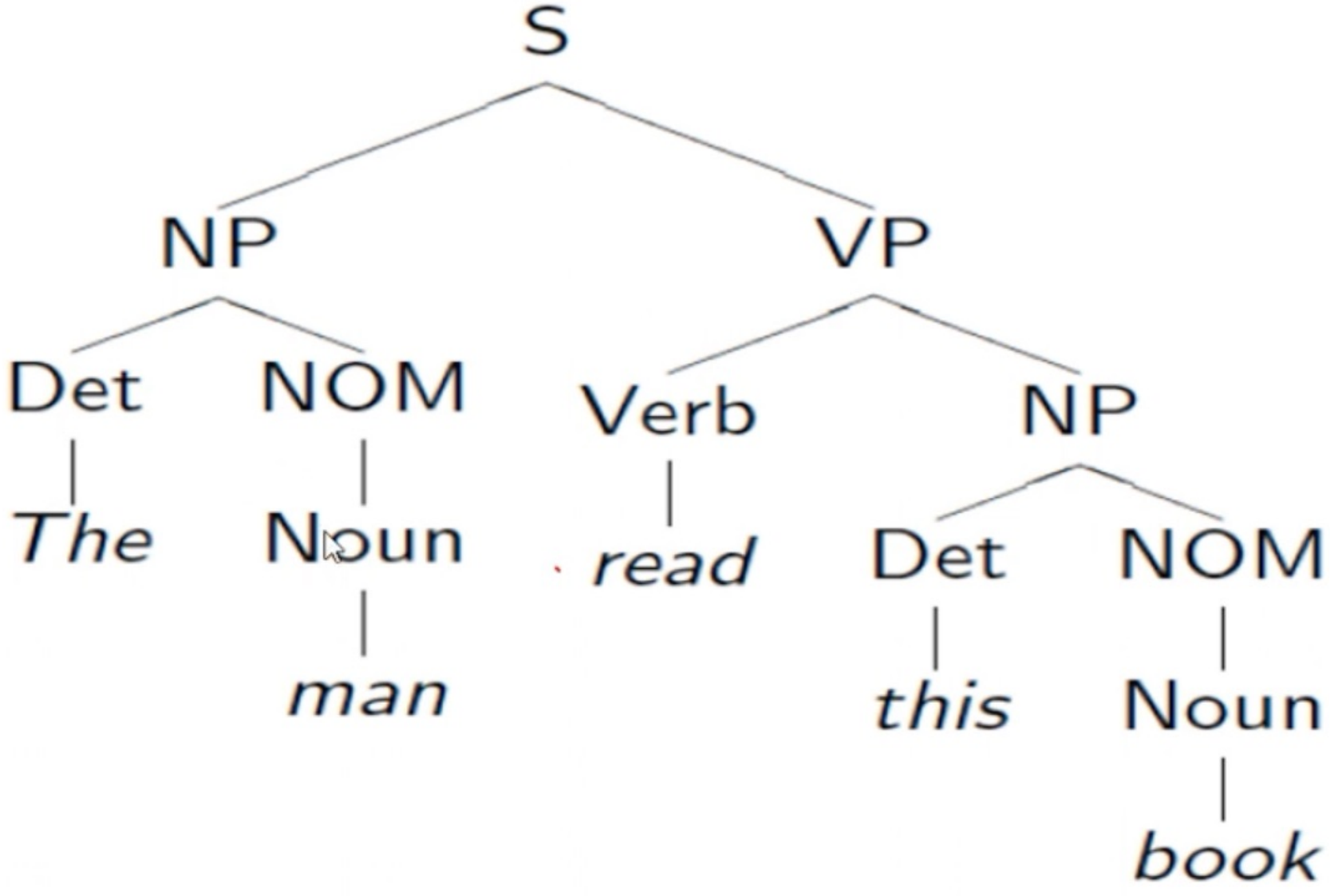
I cricket play **Incorrect statement**

In regional language like Marathi,

मी क्रिकेट खेळतो **Correct statement**

मी खेळतो क्रिकेट **Incorrect statement**

Modeling Constituency: what tool do we need?



- What is the formal tool to model this constituency?
- That is how words are arranged together?
- Which words come together? Which was do not come together?
- What groups make a sentence and what groups make a verb phrase what group make a noun phrase

Solution is Context Free Grammar

Context-Free Grammar (CFG)

- The most common approach **to modeling constituency involves using production rules**.
- These **rules specify how the symbols of a language can be grouped and arranged.**
- For example, **a noun phrase can consist of either a proper noun or a determiner followed by a nominal, where a nominal can include multiple nouns.**

Example

NP → Det Nominal

NP → ProperNoun

Nominal→ Noun | Noun Nominal

e.g. a book, the book

a book---> Det:a Nominal:Noun-->book

the book---> Det:the Nominal:Noun-->book

$G = \langle T, N, S, R \rangle$

$T = \{that,\ this,\ a,\ the,\ man,\ book,\ flight,\ meal,\ include,\ read,\ does\}$

$N = \{\text{S, NP, NOM, VP, Det, Noun, Verb, Aux}\}$

$S = S$

$R = \{$

$\text{S} \rightarrow \text{NP VP}$	$\text{Det} \rightarrow \textit{that} \mid \textit{this} \mid \textit{a} \mid \textit{the}$
$\text{S} \rightarrow \text{Aux NP VP}$	$\text{Noun} \rightarrow \textit{book} \mid \textit{flight} \mid \textit{meal} \mid \textit{man}$
$\text{S} \rightarrow \text{VP}$	$\text{Verb} \rightarrow \textit{book} \mid \textit{include} \mid \textit{read}$
$\text{NP} \rightarrow \text{Det NOM}$	$\text{Aux} \rightarrow \textit{does}$
$\text{NOM} \rightarrow \text{Noun}$	
$\text{NOM} \rightarrow \text{Noun NOM}$	
$\text{VP} \rightarrow \text{Verb}$	
$\text{VP} \rightarrow \text{Verb NP}$	

$\}$

CFGs and Grammaticality

Grammatical Sentences:

- Sentences within this set are considered grammatical.
- They adhere to the rules specified by the CFG.

Ungrammatical Sentences:

- Sentences outside this set are deemed ungrammatical.
- They do not conform to the CFG's production rules.

Terminals (T)

- **ProperNoun:** John, Mary
- **Determiner:** the, a
- **Noun:** man, woman, telescope, hill, Texas, Monday
- **Verb:** saw

Non-terminals (N)

- **S:** Sentence
- **NP:** Noun Phrase
- **VP:** Verb Phrase
- **Nom:** Nominal

Start Symbol (S)

- **S:** Sentence

Production Rules (R)

- S --> NP VP
- NP --> ProperNoun
- NP --> Determiner Nom
- Nom --> Noun
- Nom --> Noun Nom
- VP -->Verb NP
- PP --> Preposition NP
- NP --> NP PP
- VP --> VP PP

Example Derivations

Sentence: "John saw the man"

- S -> NP VP
- NP -> ProperNoun (John)
- VP -> Verb NP
- Verb (saw)
- NP -> Determiner Nom
- Determiner (the)
- Nom -> Noun (man)

What is Parsing?

The process of taking a string and a grammar and returning all possible parse trees for that string.

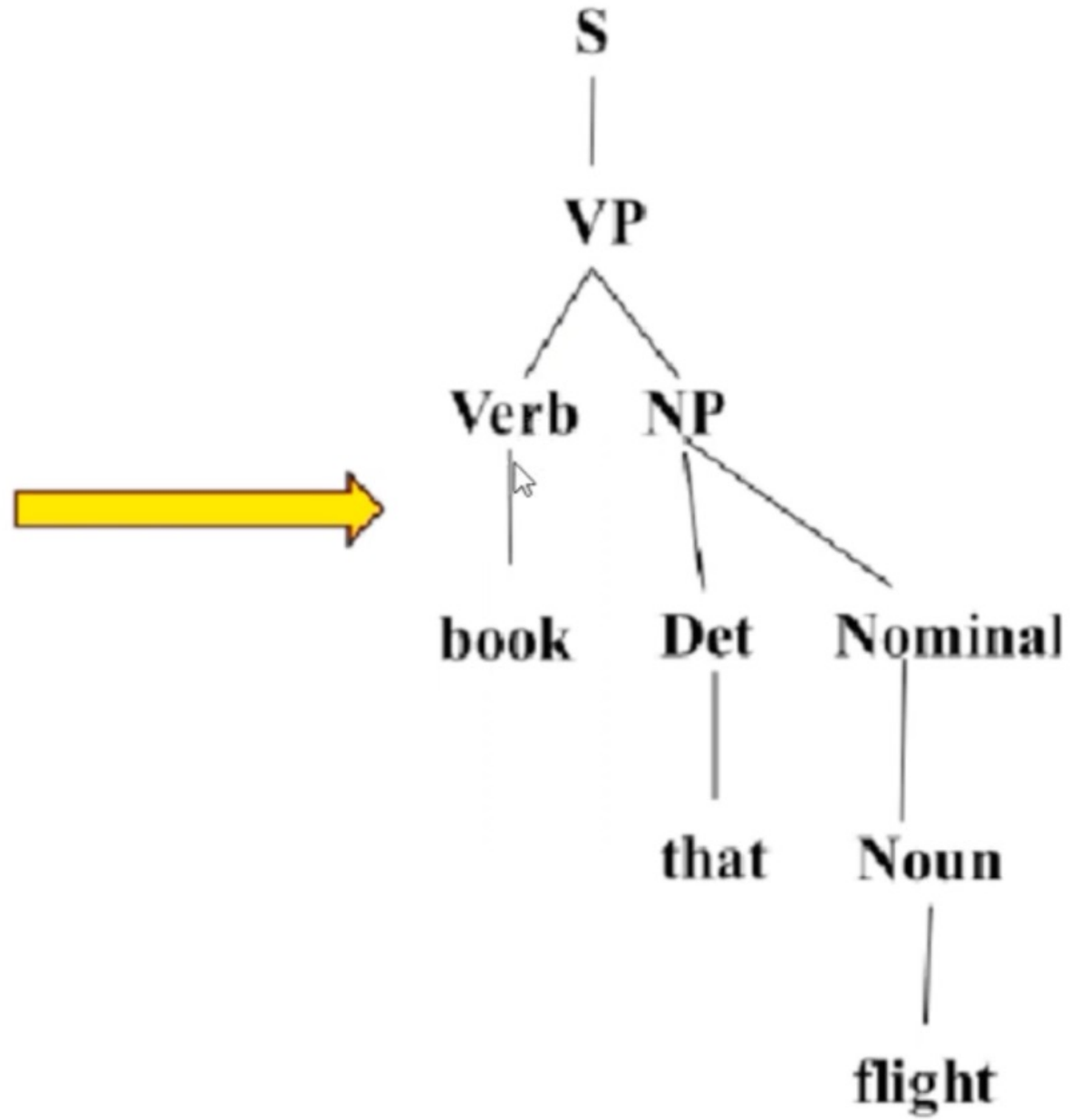
- **Top down (Goal-oriented)**
- **Bottom up (Data directed)**

Top-Down Parsing

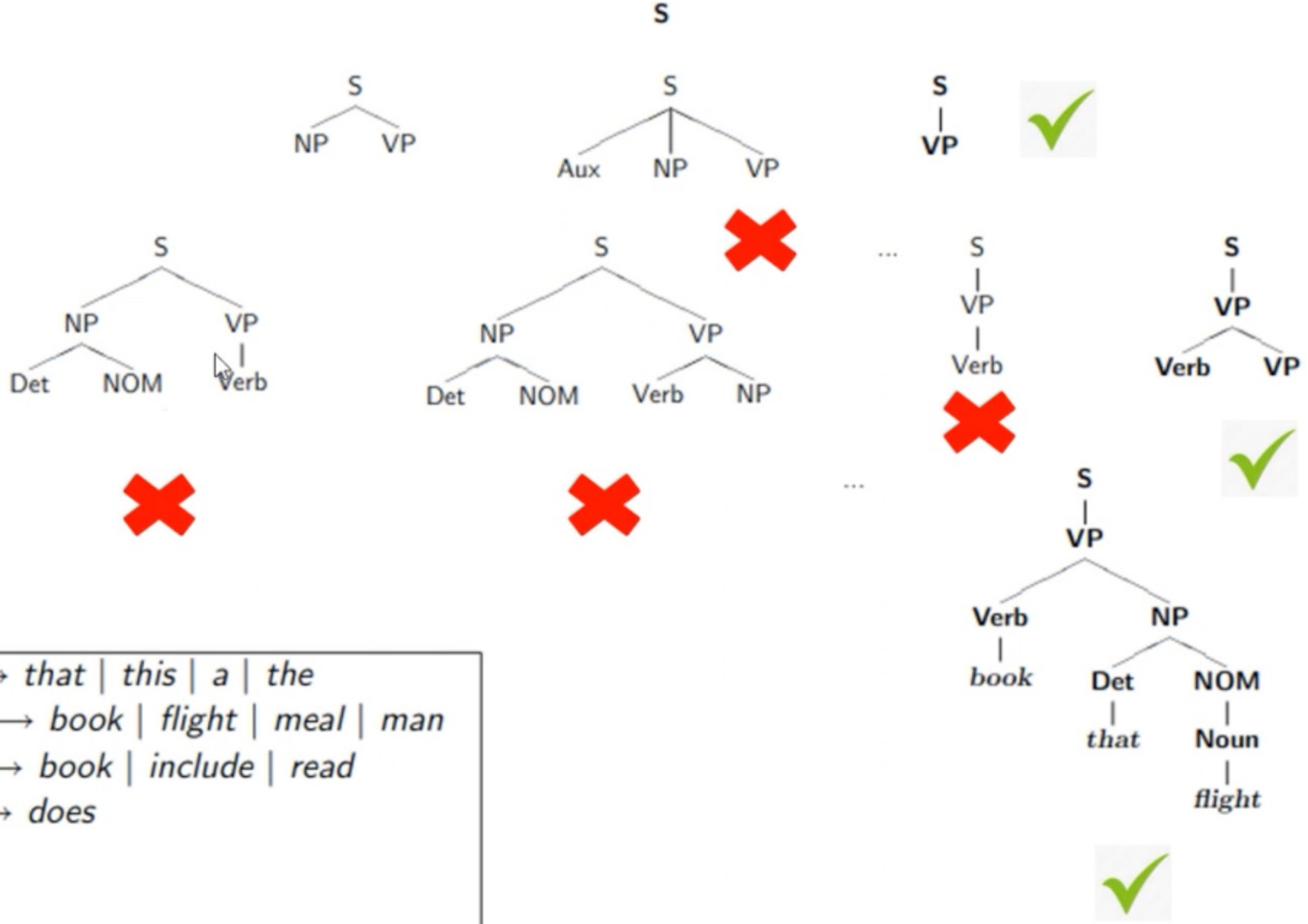
- Top-down parsing starts with the root node S and builds down to the leaves. Assuming the input can be derived from the start symbol S.
- **Initial Trees:** Identify all trees that can start with S by checking grammar rules with S on the left-hand side.
- **Grow Trees:** Expand trees downward using these rules until they reach the POS categories at the bottom.
- **Match Input:** Reject trees whose leaves do not match the input words.
- This method searches for a parse tree by starting at the top and expanding downward, verifying against the input at each step.

e.g. Early Parser, Predictive Parsing

book that flight



e.g. **book that flight**

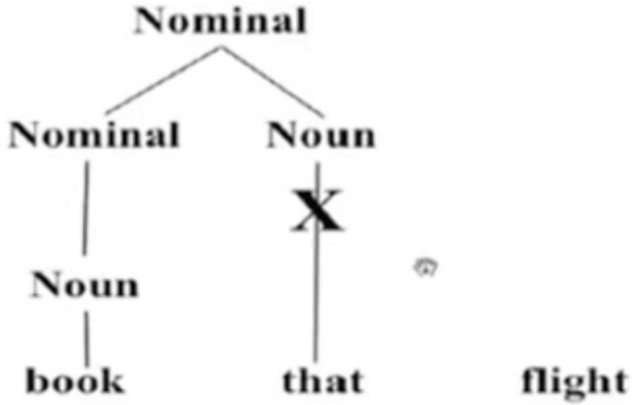


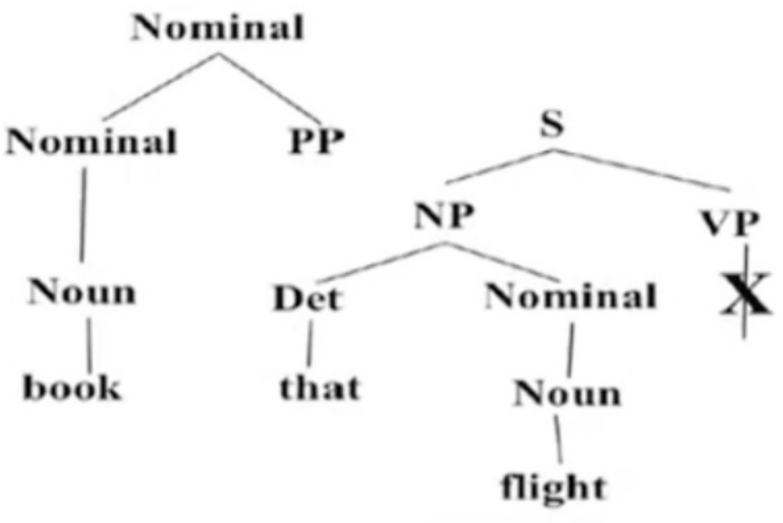
S → NP VP	Det → *that* | *this* | *a* | *the*
S → Aux NP VP	Noun → *book* | *flight* | *meal* | *man*
S → VP	Verb → *book* | *include* | *read*
NP → Det NOM	Aux → *does*
NOM → Noun	
NOM → Noun NOM	
VP → Verb	
VP → Verb NP	

- All the rules are tried in sequential manner from start to end.
- But this is not efficient method.
- Therefore there is need of parser which will efficiently parse a given statement by constructing efficient parse tree.
- If sentence is grammatically incorrect, it should discard that statement.

Bottom-Up Parsing

- The parser starts with the input words and builds trees upward.
- **Start with Words:** Begin with the words of the input.
- **Apply Rules:** Build trees by applying grammar rules one at a time.
- **Fit Rules:** Look for places in the current parse where the right-hand side of a rule can fit.
- This method constructs the parse tree from the bottom up, using the input words and grammar rules to build up to the start symbol.





S → NP VP	Det → *that* | *this* | *a* | *the*
S → Aux NP VP	Noun → *book* | *flight* | *meal* | *man*
S → VP	Verb → *book* | *include* | *read*
NP → Det NOM	Aux → *does*
NOM → Noun	
NOM → Noun NOM	
VP → Verb	
VP → Verb NP	

e.g. of Bottom Up Parsers

1. **CYK or CKY Parser:** CYK or CKY algorithm which works on **CNF (Chomsky Normal Form)** grammar only. Also generates multiple trees when statement is ambiguous.
2. **Shift Reduce Parser:** In does not require grammar to be in CNF form. But there is need to handle backtrack.
3. **PCGF (Probabilistic Context Free Grammar)**

Chomsky Normal Form

- Chomsky Normal Form(CNF) puts some constraints on the grammar rules while preserving the same language.
- The benefit is that if a grammar is in CNF, then we can avoid (not completely)the ambiguity problem during parsing.
- Chomsky Normal Form (CNF) is a simplified form of context-free grammar used in NLP parsers. In CNF, every production rule is either of the form **A → BC**, where **A, B, and C are non-terminal symbols**, or **A → a, where A is a non-terminal and a is a terminal symbol.**
- This normalization facilitates parsing algorithms, such as the CYK algorithm, by providing a uniform rule structure.
- Converting a grammar to CNF helps in systematically breaking down sentences into parse trees, making it easier to implement and optimize parsing techniques.

Bottom Up Parsers

- COCKE-YOUNGER-KASAMI (CYK or CKY Parser)
- Shift Reduce Parser
- PCGF (Probabilistic Context Free Grammar)Parser

COCKE-YOUNGER-KASAMI (CYK or CKY Parser)

- It is a **bottom-up parsing method for context-free grammars.**
- It is based on the **dynamic programming concept.**
- It requires the input grammar to be in **Chomsky Normal Form (CNF).**
- The dynamic programming algorithm requires the context-free grammar to be rendered into CNF because **it tests for possibilities to split the current sequence into two smaller sequences.**
- Any context-free grammar that does not generate the empty string can be represented in CNF using only production rules of the forms **A → BC or A → a**.
- CYK parser is efficient for parsing languages that can be represented in CNF, **systematically exploring all possible parse trees.**
- Particularly useful for **parsing ambiguous grammars and ensuring all potential parses are considered.**

CYK or CKY Algorithm

```
1:  T = n x n array of empty lists
2:  for i in 1,...,n:
3:      T[i,i] = [word[i]]
4:  for len = 0,...,n-1:
5:      while rule can be applied:
6:          if x ⟶ T[i,i+len] for some x:
7:              add x in T[i,i+len]
8:          if x ⟶ T[i,i+k] T[i+k+1,i+len] for some k,x:
9:              add x in T[i,i+len]
```

G= <T, N, S, R>

T= {a, pilot, likes, flying, planes}

N= {S, NP, VP, VBG, NNS, VBZ, DT, NN, JJ}

S= S

R={

S -> **NP VP**

VP -> **VBG NNS**

VP -> **VBZ VP**

VP -> **VBZ NP**

NP -> **DT NN**

NP -> **JJ NNS**

DT-> **a**, NN->**pilot**, VBZ->**likes**, VBG->**flying**,

JJ->**flying**, NNS->**planes**

}

CNF Rule form

A → BC or A → a.

NN: Singular Noun

NNS: Plural Noun

VBG: Gerund Verb Refers to verbs ending in -ing that function as nouns. e.g. walking, swimming etc

VBZ: Present tense, Thirdperson signular verb

JJ: Adjective

Statement: A pilot likes flying planes

a	pilot	likes	flying	planes
01	02	03	04	05
	12	13	14	15
		23	24	25
			34	35
				45

A pilot likes “flying planes”

or

A pilot likes flying “planes”

a	pilot	likes	flying	planes
DT 01	**02**	03	04	05
	NN 12	**13**	14	15
		VBZ 23	**24**	25
			VBG, JJ 34	**35**
				NNS 45

DT-->a

NN--> pilot

VBZ -->likes

VBG --> flying

JJ -->flying

NNS -->planes

CNF Rule form

A → BC or A → a.

a	pilot	likes	flying	planes
DT 01	NP 02	03	04	05
	NN 12	---- 13	14	15
		VBZ 23	--- 24	25
			VBG, JJ 34	VP, NP 35
				NNS 45

02	
(01)	(12)
DT	NN
NP (a pilot)	

13	
(12)	(23)
NN	VBZ
Nil	

24	
(23)	(34)
VBZ	VBG, JJ
Nil	

35	
(34)	(45)
VBG, JJ	NNS
VP (flying planes)	
NP (flying planes)	

S-->NP VP

VP -->VBG NNS

VP -->VBZ VP

VP-->VBZ NP

NP-->DT NN

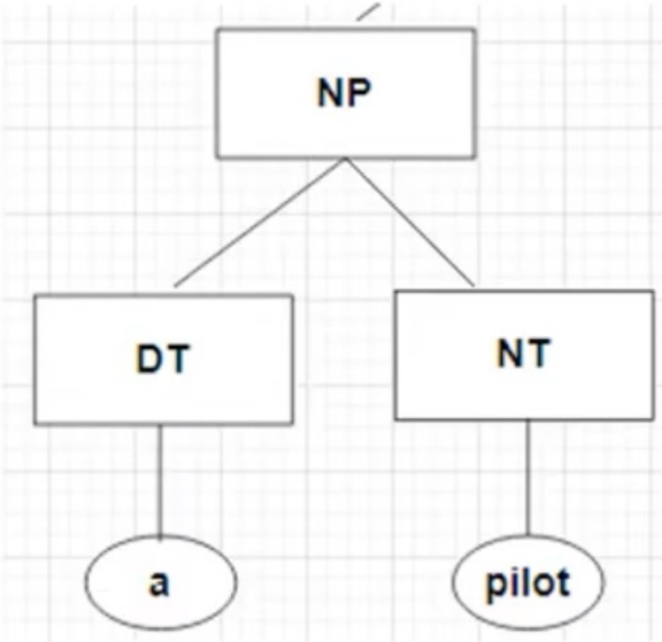
NP -->JJ NNS

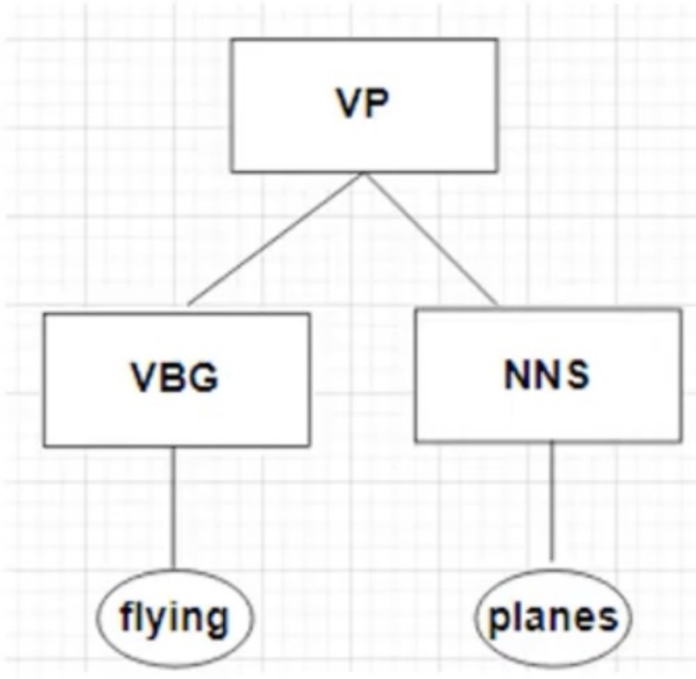
CNF Rule form

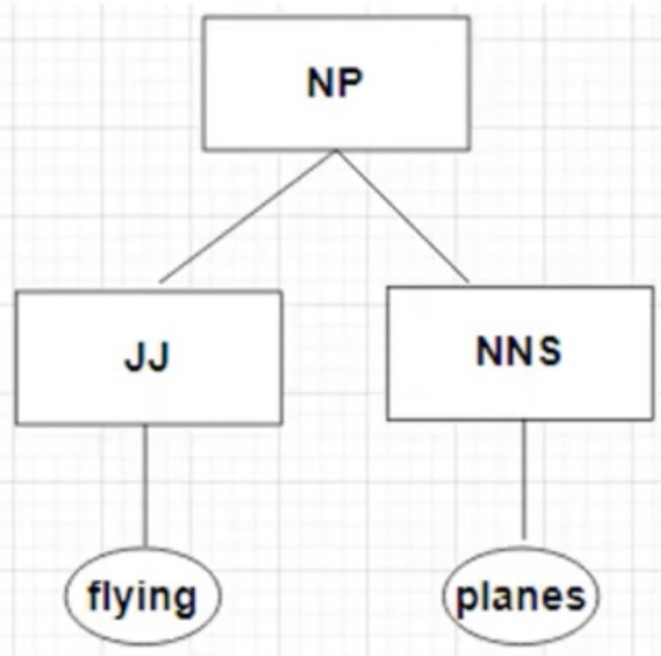
A → BC or A → a.

02	
(01)	(12)
DT	NN
NP (a pilot)	

35	
(34)	(45)
VBG, JJ	NNS
VP (flying planes)	
NP (flying planes)	







a	pilot	likes	flying	planes
DT 01	NP 02	---- 03	04	05
	NN 12	---- 13	---- 14	15
		VBZ 23	--- 24	VP1,VP2 25
			VBG, JJ 34	VP, NP 35
				NNS 45

(0 3)			
(0 1)	(1 3)	(0 2)	(2 3)
DT	Nil	NP	VBZ
Nil		Nil	

(1 4)			
(1 2)	(2 4)	(1 3)	(3 4)
NN	Nil	Nil	VBG,JJ
Nil		Nil	

(2 5)			
(2 3)	(3 5)	(2 4)	(4 5)
VBZ	VP, NP	Nil	NNS
VP1 (likes, , VP)		Nil	
VP2 (likes, NP)			

S-->NP VP

VP -->VBG NNS

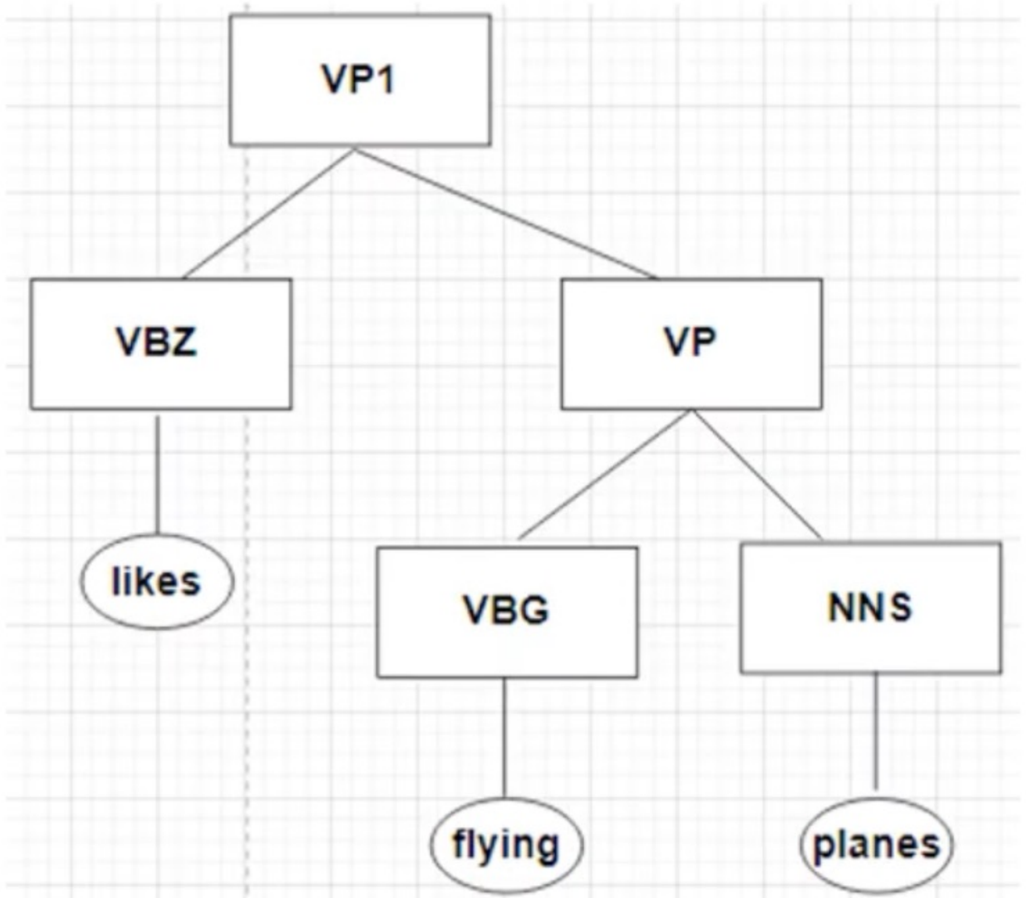
VP -->VBZ VP

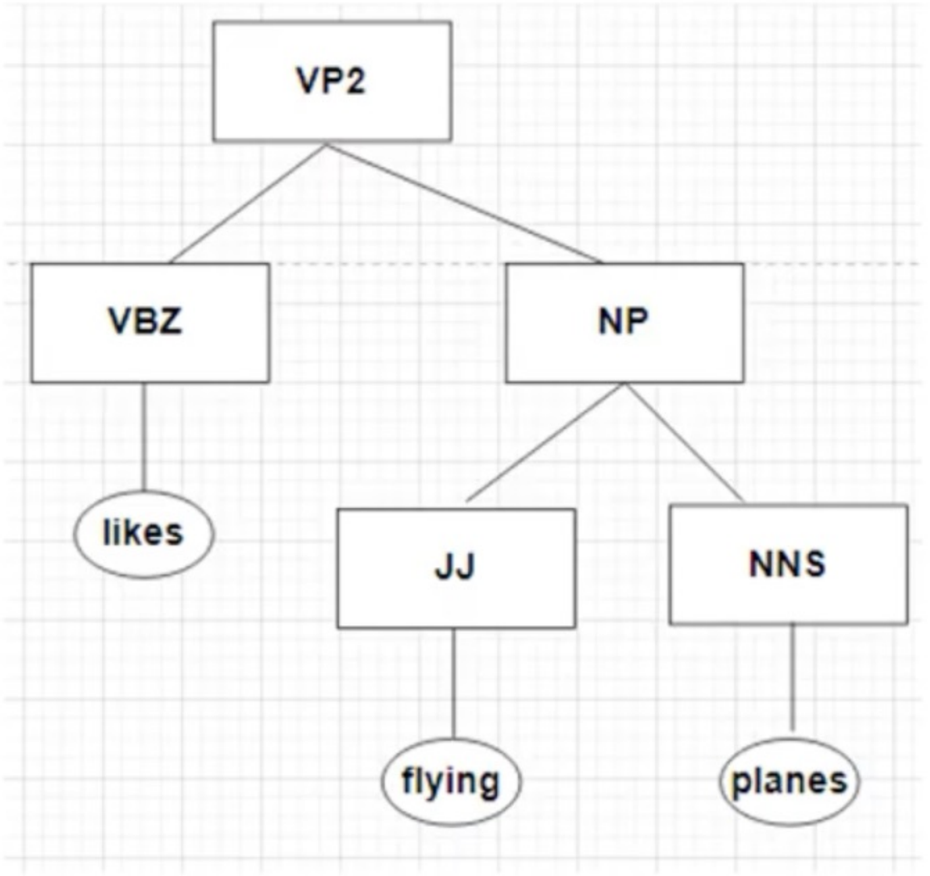
VP-->VBZ NP

NP-->DT NN

NP -->JJ NNS

(2 5)			
(2 3)	(3 5)	(2 4)	(4 5)
VBZ	VP, NP	Nil	NNS
VP1 (likes, , VP)		Nil	
VP2 (likes, NP)			





a	pilot	likes	flying	planes
DT 01	NP 02	---- 03	--- 04	05
	NN 12	---- 13	---- 14	--- 15
		VBZ 23	--- 24	VP1,VP2 25
			VBG, JJ 34	VP, NP 35
				NNS 45

(0 4)					
(0 1)	(1 4)	(0 2)	(2 4)	(0 3)	(3 4)
DT	Nil	NP	Nil	Nil	VBG, JJ
Nil		Nil		Nil	

(1 5)					
(1 2)	(2 5)	(1 3)	(3 5)	(1 4)	(4 5)
NN	VP	Nil	VP, NP	Nil	NNS
Nil		Nil		Nil	

S-->NP VP

VP -->VBG NNS

VP -->VBZ VP

VP-->VBZ NP

NP-->DT NN

NP -->JJ NNS

a	pilot	likes	flying	planes
DT 01	NP 02	---- 03	--- 04	S1, S2 05
	NN 12	---- 13	---- 14	--- 15
		VBZ 23	--- 24	VP1,VP2 25
			VBG, JJ 34	VP, NP 35
				NNS 45

(0 5)							
(0 1)	(1 5)	(0 2)	(2 5)	(0 3)	(3 5)	(0 4)	(4 5)
DT	Nil	NP	VP1, VP2	Nil	VP, NP	Nil	NNS
Nil		S1 (NP, VP1)		Nil		Nil	
		S2 (NP, VP2)					

S-->NP VP

VP -->VBG NNS

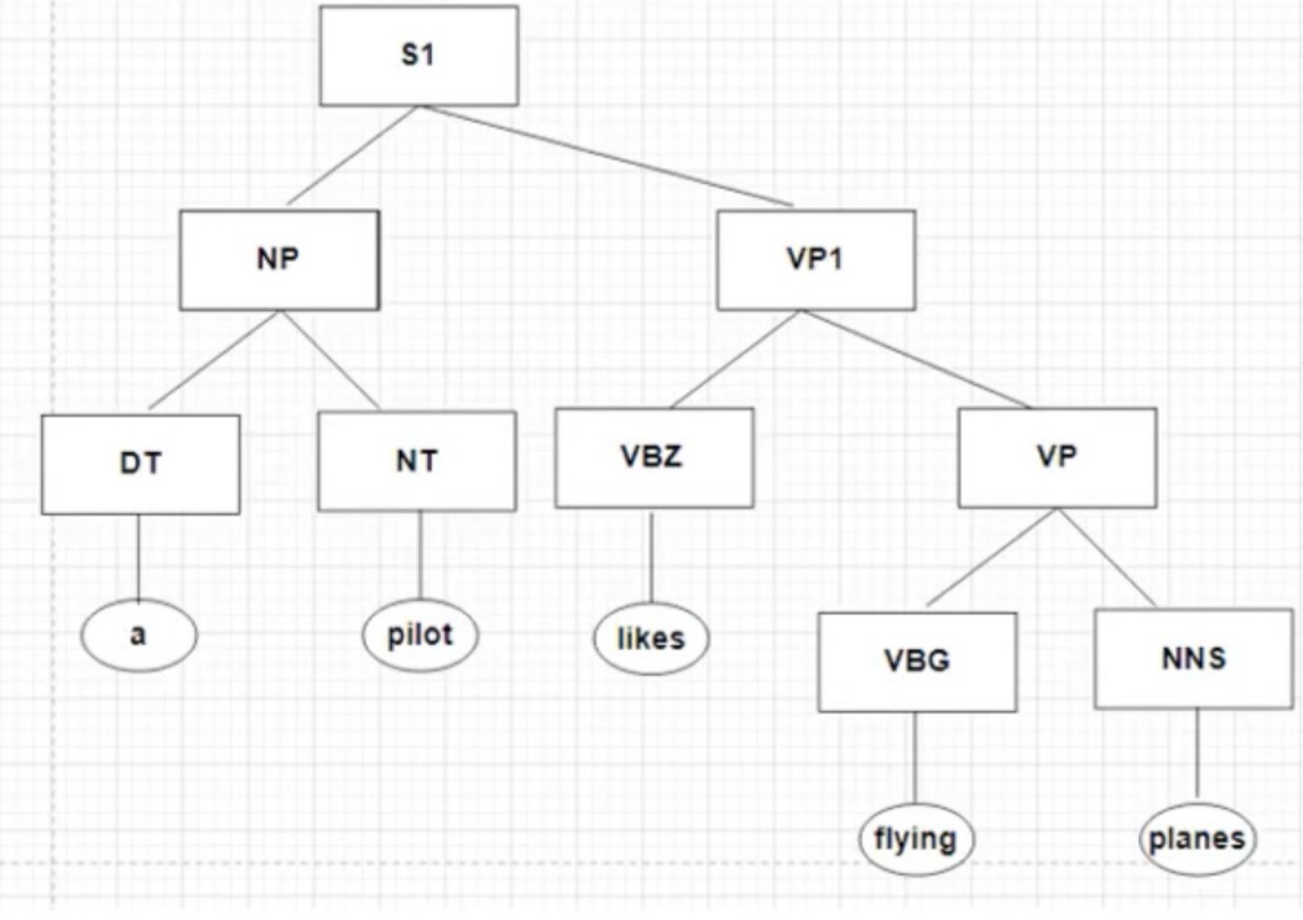
VP -->VBZ VP

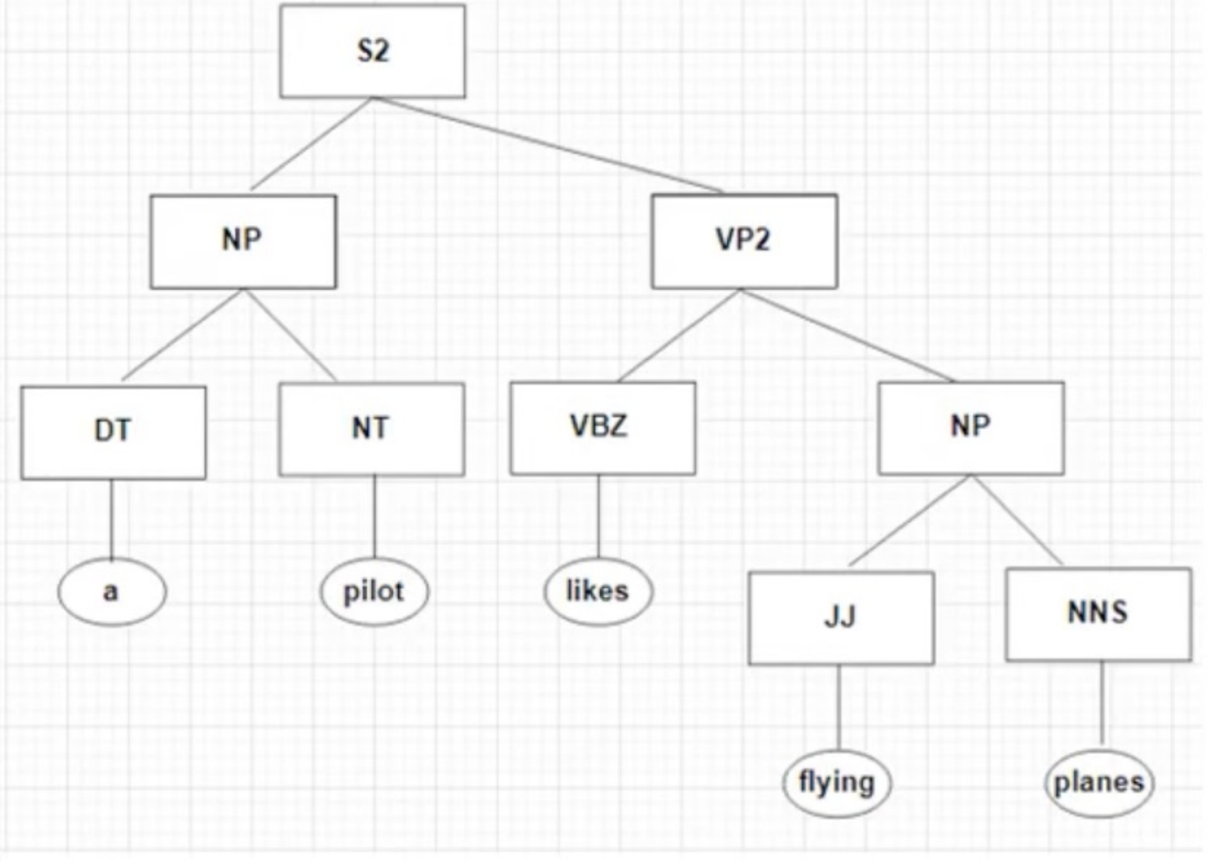
VP-->VBZ NP

NP-->DT NN

NP -->JJ NNS

(0 5)							
(0 1)	(1 5)	(0 2)	(2 5)	(0 3)	(3 5)	(0 4)	(4 5)
DT	Nil	NP	VP1, VP2	Nil	VP, NP	Nil	NNS
Nil		S1 (NP, VP1)		Nil		Nil	
		S2 (NP, VP2)					





G= <T, N, S, R>

T= {time, flies, like, an, arrow}

N= {S, NP, VP, Vst, PP, V, Det, N, P}

S= S

R={

S -> **NP VP**

S -> **Vst NP**

S -> **S PP**

VP -> **V NP**

VP -> **VP PP**

NP -> **Det N**

NP -> **NP PP**

NP -> **NP NP**

PP -> **P NP**

NP-> **time**, Vst->**time**, NP->**flies**, VP->**flies**,

P->**like**, V->**like**, Det->**an**, N->**arrow**

}

Statement: time files like an arrow

NP → time	S → NP VP
Vst → time	S → Vst NP
NP → flies	S → S PP
VP → flies	VP → V NP
P → like	VP → VP PP
V → like	NP → Det N
Det → an	NP → NP PP
N → arrow	NP → NP NP
	PP → P NP

time	flies	like	an	arrow
NP, Vst 01	02	03	04	05
	NP, VP 12	NP 13	14	15
		P, V 23	24	25
			Det 34	PP 35
				N 45

NP → time

Vst → time

NP → flies

VP → flies

P → like

V → like

Det → an

N → arrow

time	flies	like	an	arrow
NP, Vst 01	S1, S2, NP 02	03	04	05
	NP, VP 12	--- 13	14	15
		P, V 23	---- 24	25
			Det 34	NP 35
				N 45

02	
(01)	(12)
NP, Vst	NP, VP
S1(NP, VP) (time flies)	
S2(Vst, NP)(time flies)	
NP(NP, NP) (time, flies)	

13	
(12)	(23)
NP, VP	P,V
Nil	

24	
(23)	(34)
P,V	Det
Nil	

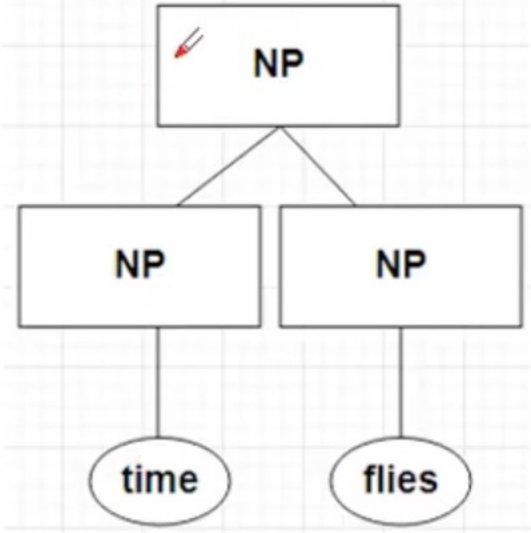
35	
(34)	(45)
Det	N
NP (an arrow)	

$$S \rightarrow NP\ VP \mid S \rightarrow Vst\ NP \mid S \rightarrow S\ PP$$

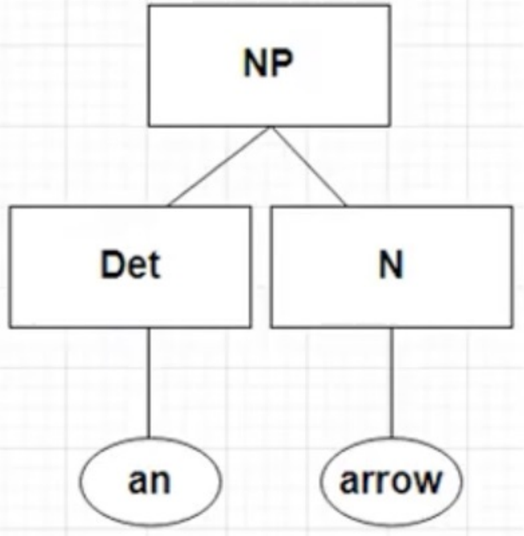
$$VP \rightarrow V\ NP \mid VP \rightarrow VP\ PP \mid PP \rightarrow P\ NP$$

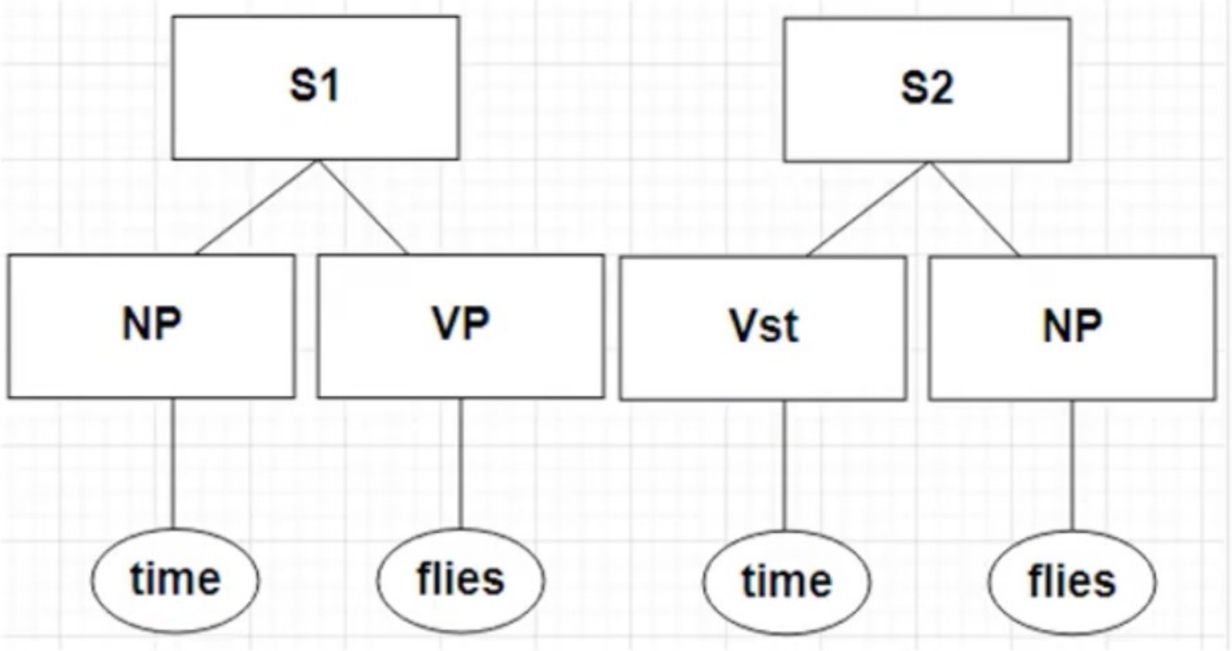
$$NP \rightarrow Det\ N \mid NP \rightarrow NP\ PP \mid NP \rightarrow NP\ NP$$

02	
(01)	(12)
NP, Vst	NP, VP
S1(NP, VP) (time flies)	
S2(Vst, NP)(time flies)	
NP(NP, NP) (time, flies)	



35	
(34)	(45)
Det	N
NP (an arrow)	





time	flies	like	an	arrow
NP, Vst 01	S1, S2, NP 02	--- 03	04	05
	NP, VP 12	--- 13	--- 14	15
		P, V 23	---- 24	PP, VP 25
			Det 34	NP 35
				N 45

(0 3)			
(0 1)	(1 3)	(0 2)	(2 3)
NP, Vst	Nil	S1,S2,NP	P, V
Nil		Nil	

(1 4)			
(1 2)	(2 4)	(1 3)	(3 4)
Np, VP	Nil	Nil	Det
Nil		Nil	

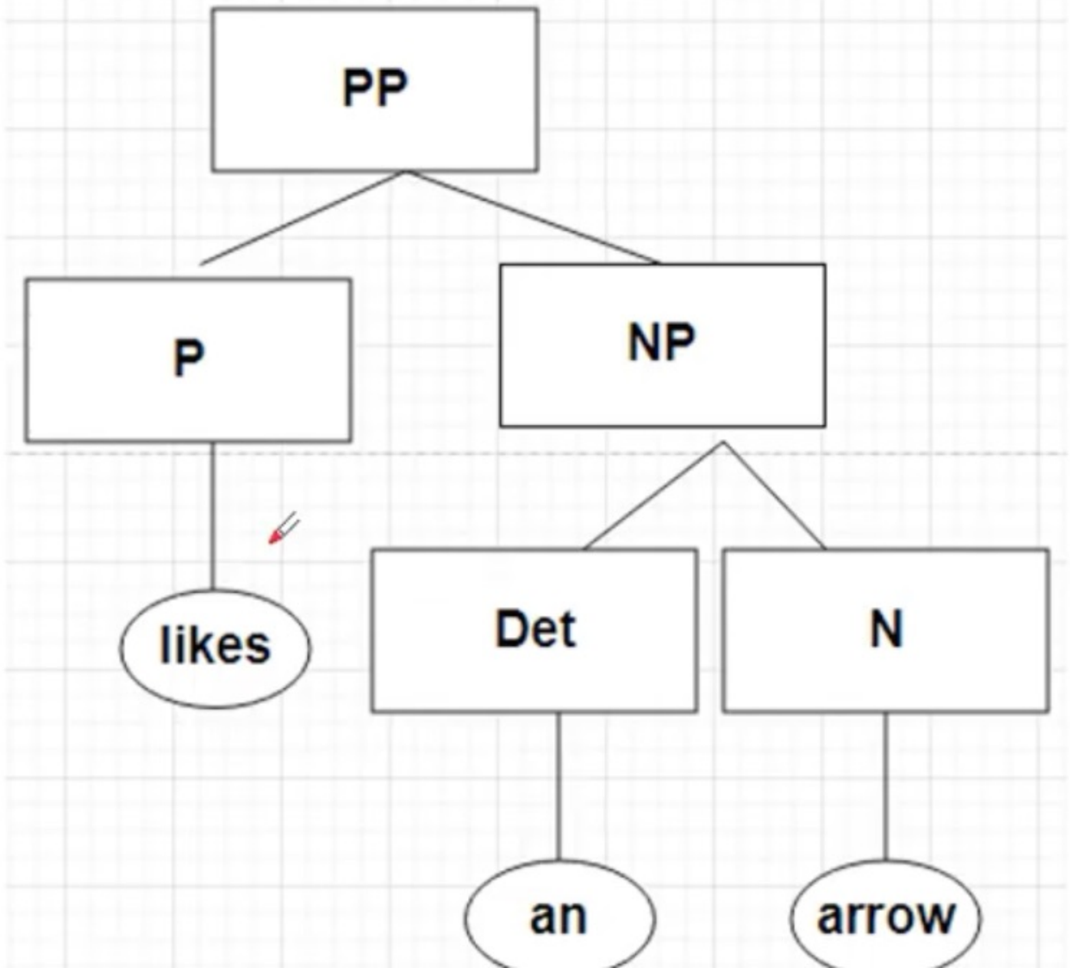
(2 5)			
(2 3)	(3 5)	(2 4)	(4 5)
P, V	NP	Nil	N
PP (P, NP)		Nil	
VP (V, NP)			

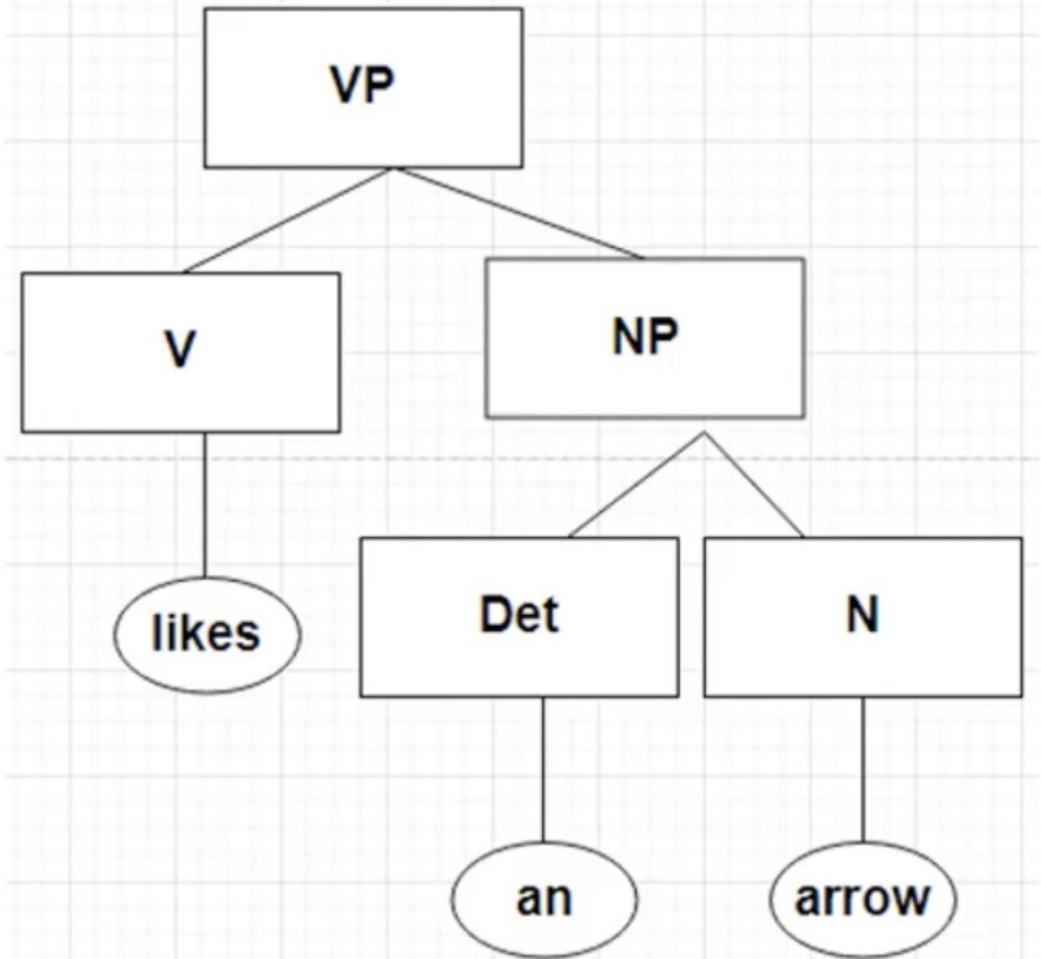
$S \rightarrow NP\ VP$ | $VP \rightarrow V\ NP$ | $NP \rightarrow Det\ N$

$S \rightarrow Vst\ NP$ | $VP \rightarrow VP\ PP$ | $NP \rightarrow NP\ PP$

$S \rightarrow S\ PP$ | $PP \rightarrow P\ NP$ | $NP \rightarrow NP\ NP$

(2 5)			
(2 3)	(3 5)	(2 4)	(4 5)
P, V	NP	Nil	N
PP (P, NP)		Nil	
VP (V, NP)			





time	flies	like	an	arrow
NP, Vst 01	S1, S2, NP 02	--- 03	04	05
	NP, VP 12	--- 13	--- 14	NP, S3, VP 15
		P, V 23	---- 24	PP, VP 25
			Det 34	NP 35
				N 45

(0 4)					
(0 1)	(1 4)	(0 2)	(2 4)	(0 3)	(3 4)
Np, Vst	Nil	S1,S2,Np	Nil	Nil	Det
Nil		Nil		Nil	

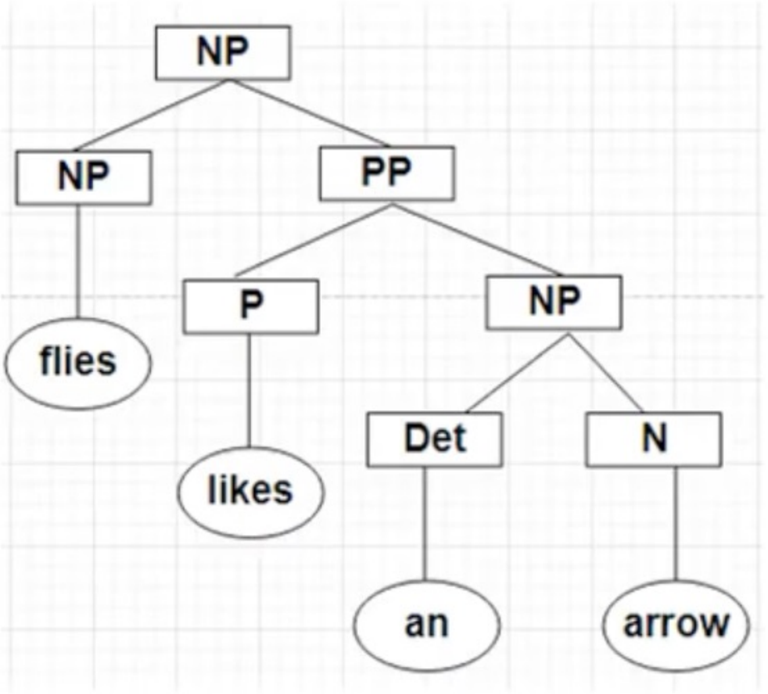
(1 5)					
(1 2)	(2 5)	(1 3)	(3 5)	(1 4)	(4 5)
NP, VP	PP, VP	Nil	NP	Nil	N
NP(NP, PP)		Nil		Nil	
S3(NP, PP)					
VP(VP,PP)					

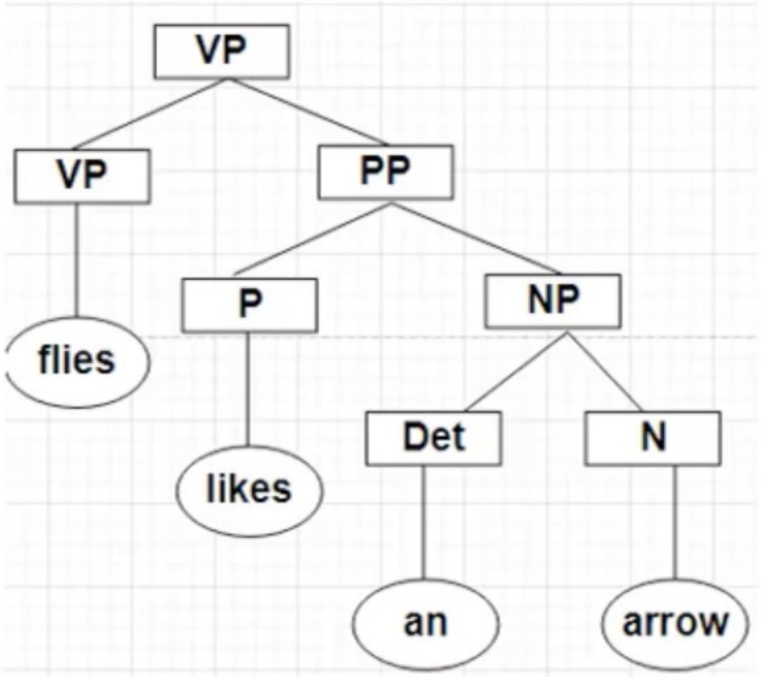
$S \rightarrow NP\ VP$ | $VP \rightarrow V\ NP$ | $NP \rightarrow Det\ N$

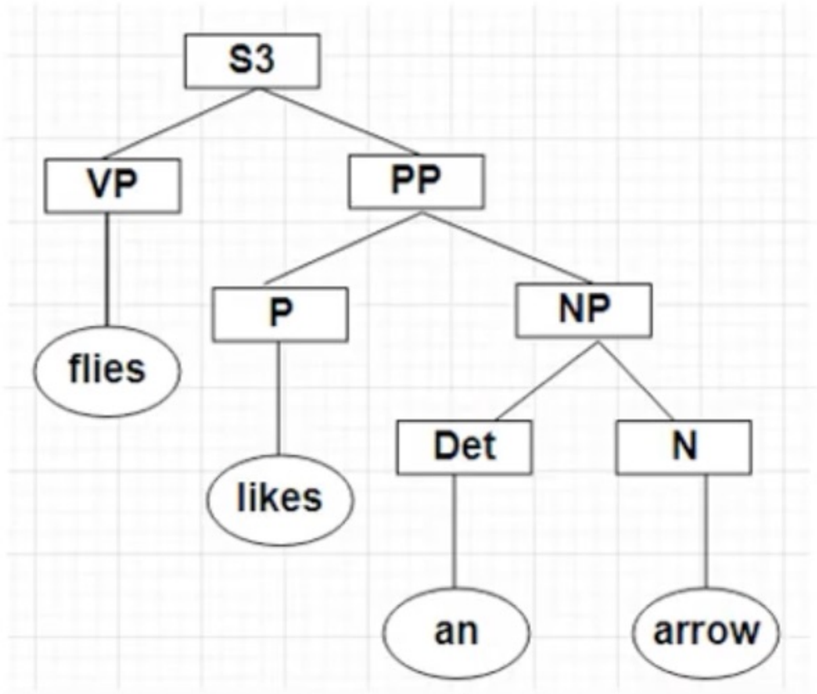
$S \rightarrow Vst\ NP$ | $VP \rightarrow VP\ PP$ | $NP \rightarrow NP\ PP$

$S \rightarrow S\ PP$ | $PP \rightarrow P\ NP$ | $NP \rightarrow NP\ NP$

(1 5)					
(1 2)	(2 5)	(1 3)	(3 5)	(1 4)	(4 5)
NP, VP	PP, VP	Nil	NP	Nil	N
NP(NP, PP)		Nil		Nil	
S3(NP, PP)					
VP(VP,PP)					







time	flies	like	an	arrow
NP, Vst 01	S1, S2, NP 02	--- 03	04	NP, S4, S5 05
	NP, VP 12	--- 13	--- 14	NP, S3, VP 15
		P, V 23	---- 24	PP, VP 25
			Det 34	NP 35
				N 45

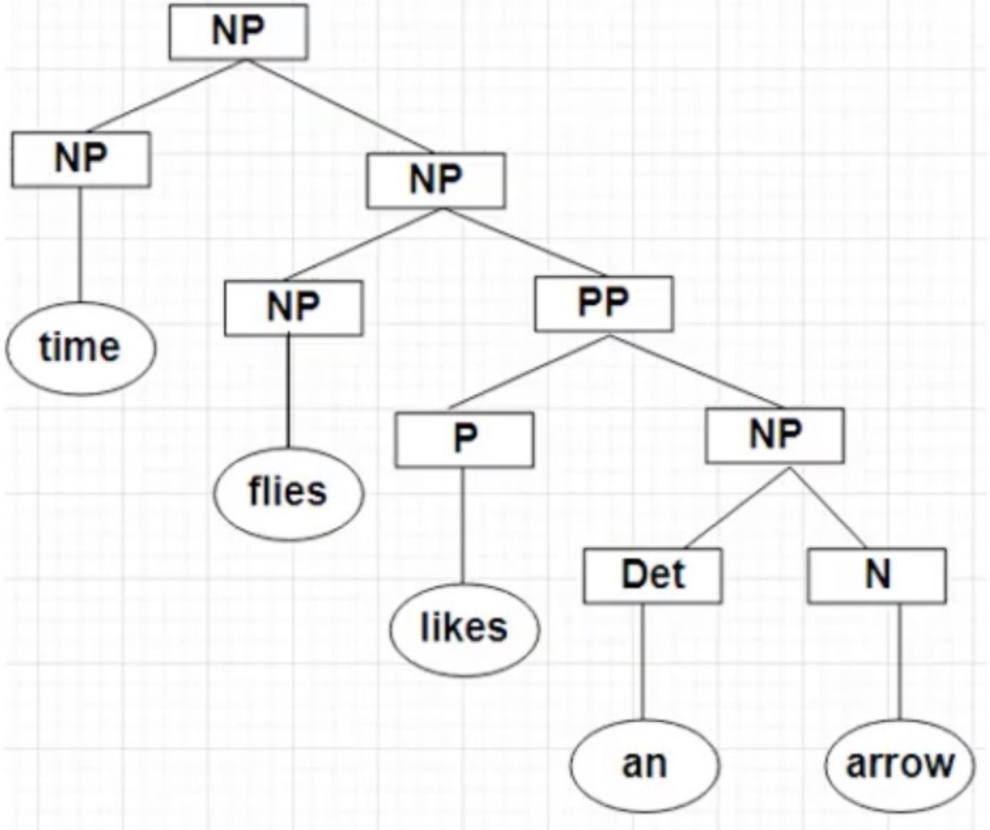
			(0 5)				
(0 1)	(1 5)	(0 2)	(2 5)	(0 3)	(3 5)	(0 4)	(4 5)
NP, Vst	NP, S, VP	S1,S2,NP	PP,VP	Nil	NP	Nil	N
NP(NP, NP)		NP(NP, PP)		Nil		Nil	
S4(NP,VP)							
S5(Vst, NP)							

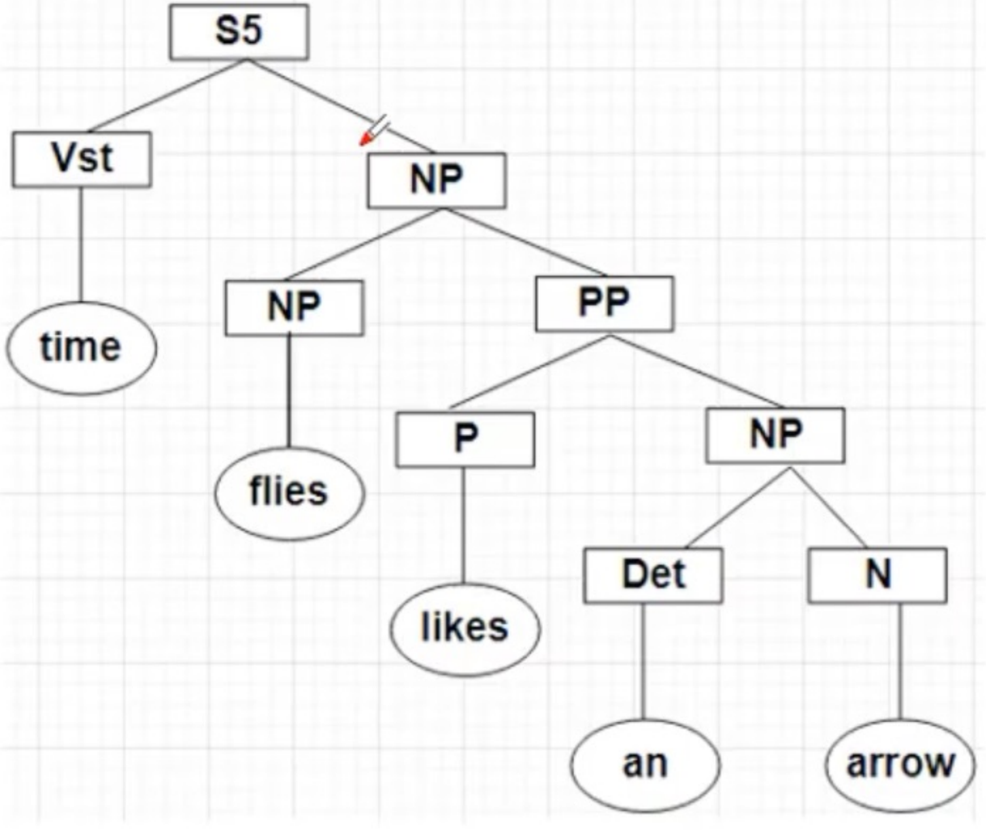
$S \rightarrow NP\ VP$ | $VP \rightarrow V\ NP$ | $NP \rightarrow Det\ N$

$S \rightarrow Vst\ NP$ | $VP \rightarrow VP\ PP$ | $NP \rightarrow NP\ PP$

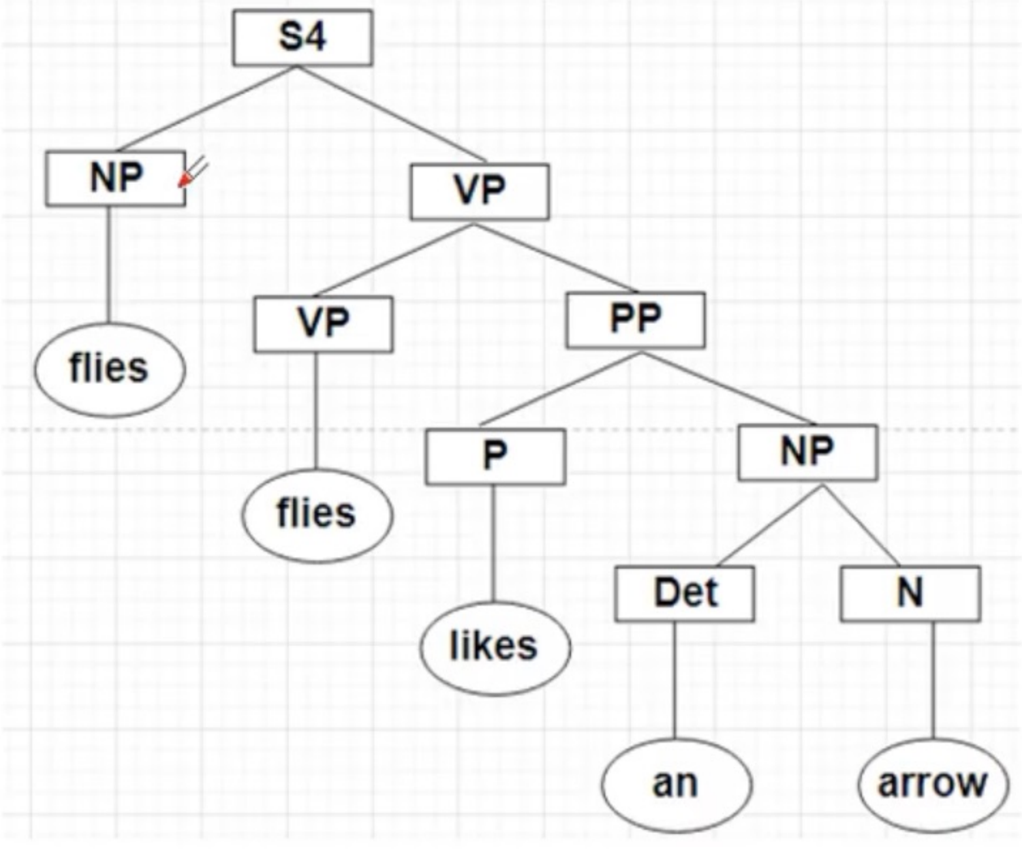
$S \rightarrow S\ PP$ | $PP \rightarrow P\ NP$ | $NP \rightarrow NP\ NP$

			(0 5)				
(0 1)	(1 5)	(0 2)	(2 5)	(0 3)	(3 5)	(0 4)	(4 5)
NP, Vst	NP, S, VP	S1,S2,NP	PP,VP	Nil	NP	Nil	N
NP(NP, NP)		NP(NP, PP)		Nil		Nil	
S4(NP,VP)							
S5(Vst, NP)							





			(0 5)				
(0 1)	(1 5)	(0 2)	(2 5)	(0 3)	(3 5)	(0 4)	(4 5)
NP, Vst	NP, S, VP	S1,S2,NP	PP,VP	Nil	NP	Nil	N
NP(NP, NP)		NP(NP, PP)		Nil		Nil	
S4(NP,VP)							
S5(Vst, NP)							



Bottom Up Parsers

- PCGF (Probabilistic Context Free Grammar)Parser

Problem with CYK Parser

Problem: How can we differentiate between multiple possible parses of a given sentence?

The goal is to select the parse tree that best represents the intended meaning.

Possible Solutions:

- Delegate the task to Semantic Processing.
- Implement a probabilistic model to evaluate and rank the possible parse trees based on their likelihoods, and choose the most probable one. By assigning probabilities to grammar rules, we can create such a model.
- The probabilistic context-free grammar (PCFG) is the most widely used probabilistic grammar framework, enhancing traditional context-free grammars by attaching a probability to each rule.

PCGF (Probabilistic Context Free Grammar)Parser

- PCFG parsers are used in natural language processing (NLP) **to parse sentences and determine their most likely syntactic structures.**
- A Probabilistic Context-Free Grammar (PCFG) parser is an **extension of the traditional context-free grammar (CFG) parser, where each production rule is assigned a probability.**
- These probabilities **represent the likelihood of using a particular production rule in the derivation of a sentence.**

Grammar and Probabilities:

A PCFG consists of a set of production rules, similar to a CFG, but **each rule is accompanied by a probability.**

The sum of probabilities of all rules with the **same left-hand side non-terminal equals 1.**

Parsing Process:

The parser generates possible parse trees for a given sentence.

It calculates the **probability of each parse tree by multiplying the probabilities of the production rules used to generate the tree.**

The parse tree with the **highest probability is chosen as the most likely syntactic structure of the sentence.**

Assigning Probabilities:

Probabilities can be assigned to production rules based on their frequencies in **a corpus of text.** For example, if a non-terminal A has two production rules $A \rightarrow B$ & A→C the probabilities can be determined as follows:

$$P(A \rightarrow B) = \frac{\text{Count}(A \rightarrow B)}{\text{Count}(A)}$$

$$P(A \rightarrow C) = \frac{\text{Count}(A \rightarrow C)}{\text{Count}(A)}$$

where

Count (A) : Total number of times A appears in the corpus

Count($A \rightarrow B$): Number of times the production $A \rightarrow B$ is used.

$S \rightarrow NP\ VP$ (0.9)

$S \rightarrow VP$ (0.1)

$NP \rightarrow Det\ N$ (0.8)

$NP \rightarrow N$ (0.2)

$VP \rightarrow V\ NP$ (0.7)

$VP \rightarrow V$ (0.3)

$Det \rightarrow the$ (0.6)

$Det \rightarrow a$ (0.4)

$N \rightarrow cat$ (0.5)

$N \rightarrow dog$ (0.5)

$V \rightarrow chases$ (0.5)

$V \rightarrow sees$ (0.5)

Statement: Book book

$S \rightarrow NP\ VP$ [0.6] $VP \rightarrow Verb$ [0.3]

$S \rightarrow VP$ [0.4] $VP \rightarrow Verb\ NP$ [0.7]

$NP \rightarrow Noun$ [1.0]

$Noun \rightarrow book$ [0.1]

$Verb \rightarrow book$ [0.2]

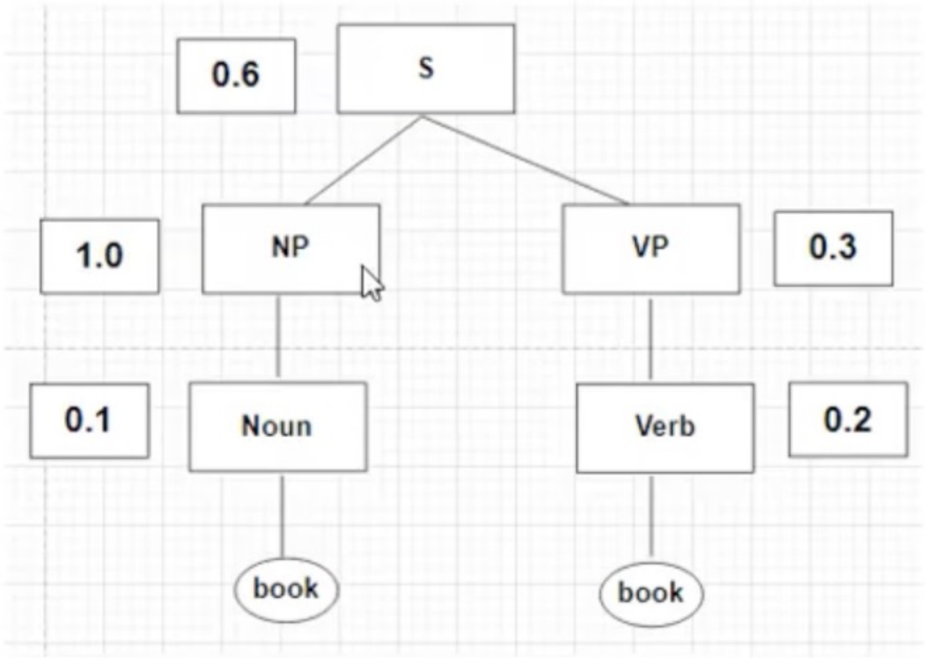
book	book
Noun(0.1) Verb(0.2) 01	S1 S2 02
	Noun(0.1) Verb(0.2) 12

02		
(01)	(12)	
Noun(0.1), Verb(0.2)	Noun(0.1), Verb(0.2)	
S1 (Noun(0.1), Verb(0.2)): NP(1.0)*VP(0.3)*S(0.6)	**0.1*0.2*1*0.3**	**0.0036**
S2(Noun(0.1), Verb(0.2)): NP(1.0)*VP(0.7)*S(0.4)	**0.1*0.2*1.0*0.**	**0.0056**
(Noun, Noun)		
(Verb, Verb)		

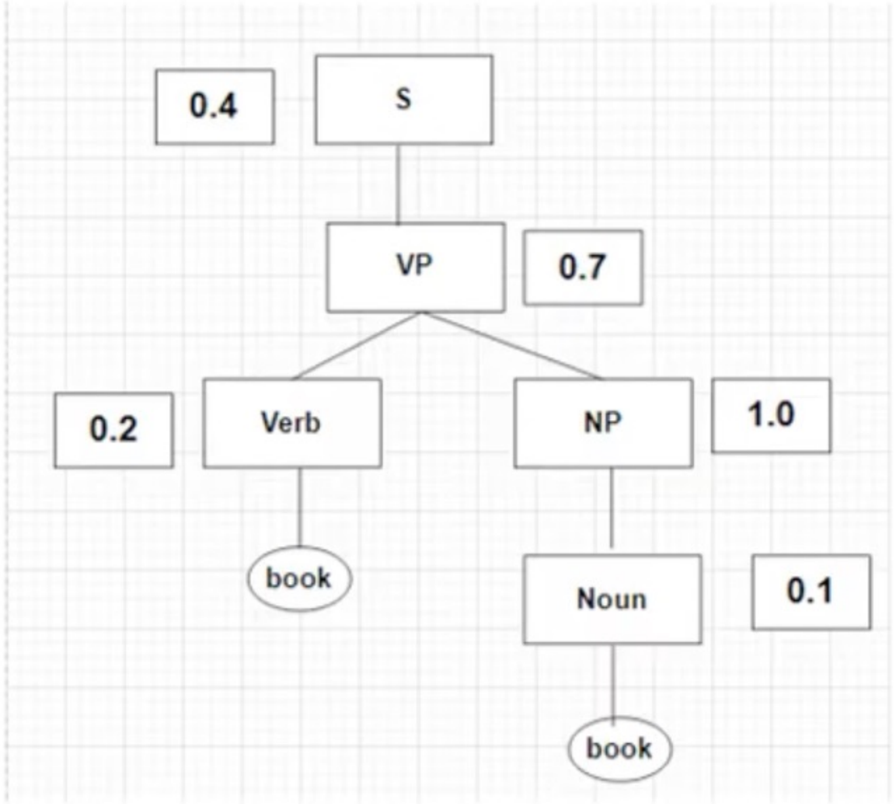
Two Possibilities

Correct :Book (Verb) book (Noun)

Incorrect :Book (Noun) book (Verbs)



S=0.1*0.2*1.0*0.3*0.6=0.0036



S=0.2*0.1*1.0*0.7*0.4=0.0056

$S \rightarrow NP\ VP$ [0.6] $VP \rightarrow Verb$ [0.3]

$S \rightarrow VP$ [0.4] $VP \rightarrow Verb\ NP$ [0.7]

$NP \rightarrow Noun$ [1.0]

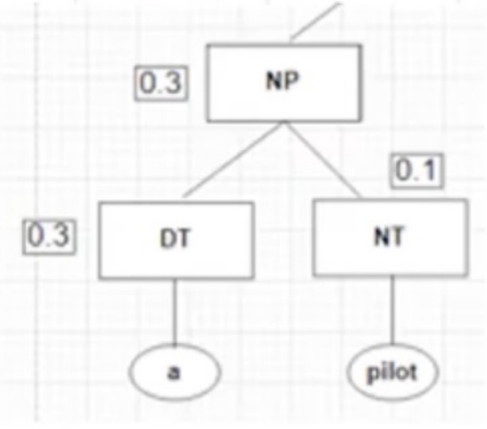
$Noun \rightarrow book$ [0.1]

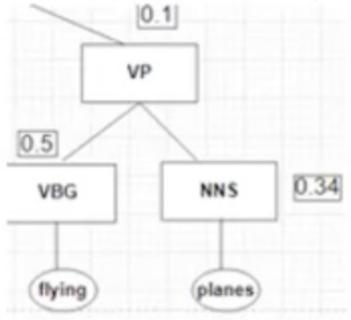
Statement: A pilot likes flying planes

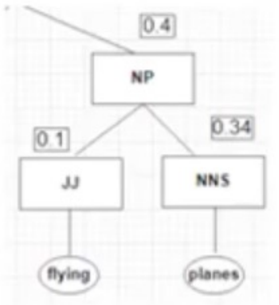
a	pilot	likes	flying	planes
DT (0.3) 01	NP (0.009) 02	03	04	05
	NN (0.1) 12	--- 13	14	15
		VBZ(0.4) 23	--- 24	25
			VBG (0.5), JJ (0.1) 34	NP (0.0136), VP (0.017) 35
				NNS (0.34) 45

Rules	Probability
R={	
S -> NP VP	1.0
VP -> VBG NNS	0.1
VP -> VBZ VP	0.1
VP -> VBZ NP	0.3
NP -> DT NN	0.3
NP -> JJ NNS	0.4
DT-> a	0.3
NN->pilot	0.1
VBZ->likes	0.4
VBG->flying	0.5
JJ->flying	0.1
NNS->planes	0.34
}	

NP	DT	NN	
0.3	0.3	0.1	0.009
NP	**JJ**	**NNS**	
0.4	0.1	0.34	0.0136
VP	**VBG**	**NNS**	
0.1	0.5	0.34	0.017



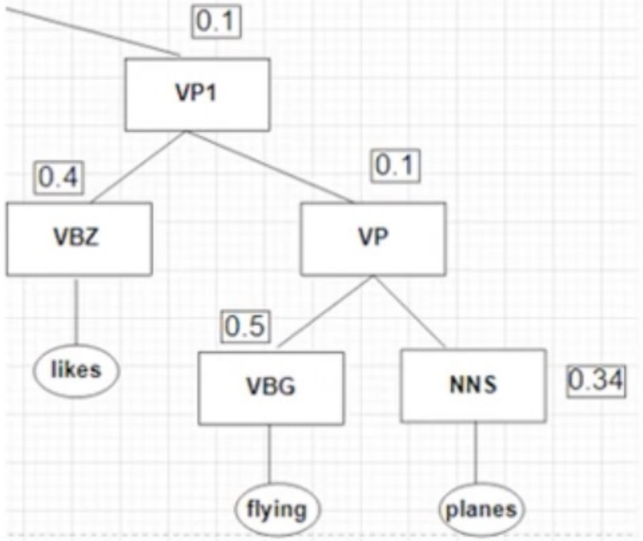


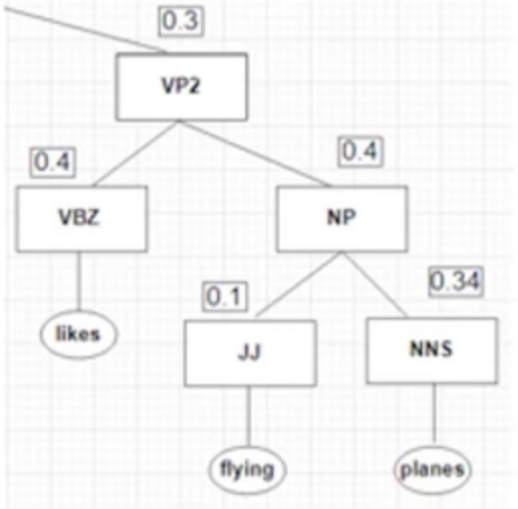


a	pilot	likes	flying	planes
DT (0.3) 01	**NP (0.009)** 02	03	04	05
	NN (0.1) 12	--- 13	14	15
		VBZ(0.4) 23	--- 24	**VP1 (0.001632) ,VP2 (0.00068)** 25
			VBG (0.5), JJ (0.1) 34	**NP (0.0136), VP (0.017)** 35
				NNS (0.34) 45

Rules	Probability
R={	
S -> NP VP	1.0
VP -> VBG NNS	0.1
VP -> VBZ VP	0.1
VP -> VBZ NP	0.3
NP -> DT NN	0.3
NP -> JJ NNS	0.4
DT-> a	0.3
NN->pilot	0.1
VBZ->likes	0.4
VBG->flying	0.5
JJ->flying	0.1
NNS->planes	0.34
}	

VP1	NP	VBZ	
0.3	0.0136	0.4	0.001632
VP2	**VP**	**VBZ**	
0.1	0.017	0.4	0.00068

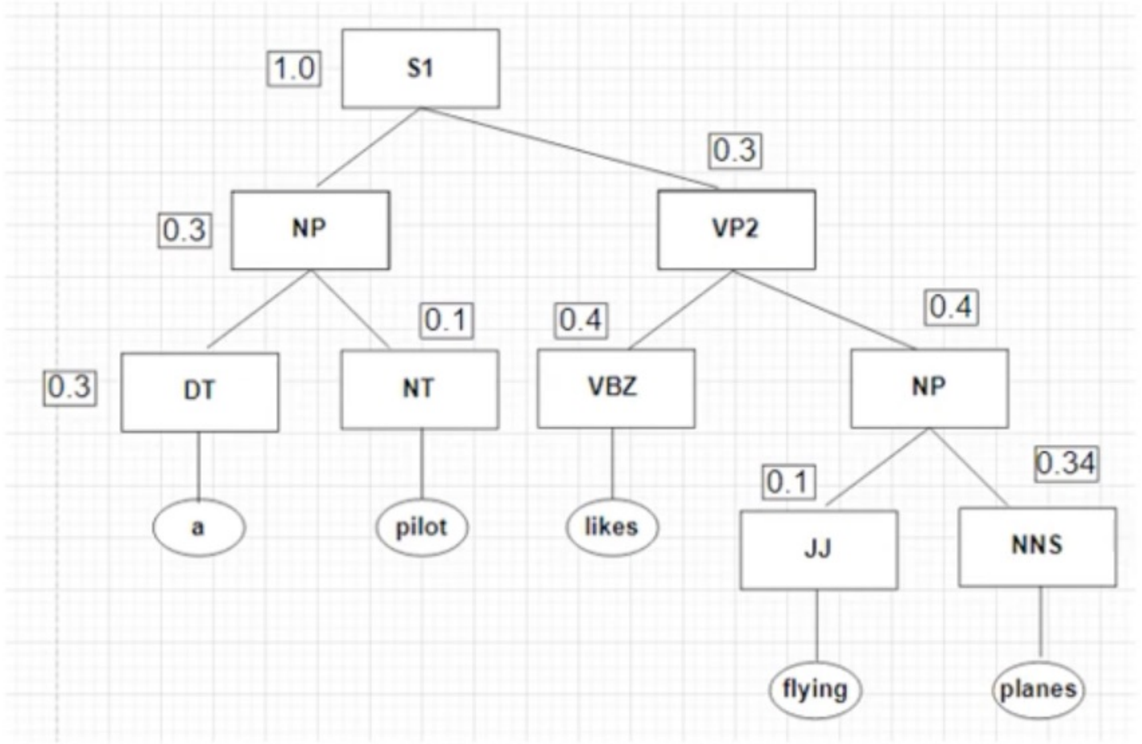


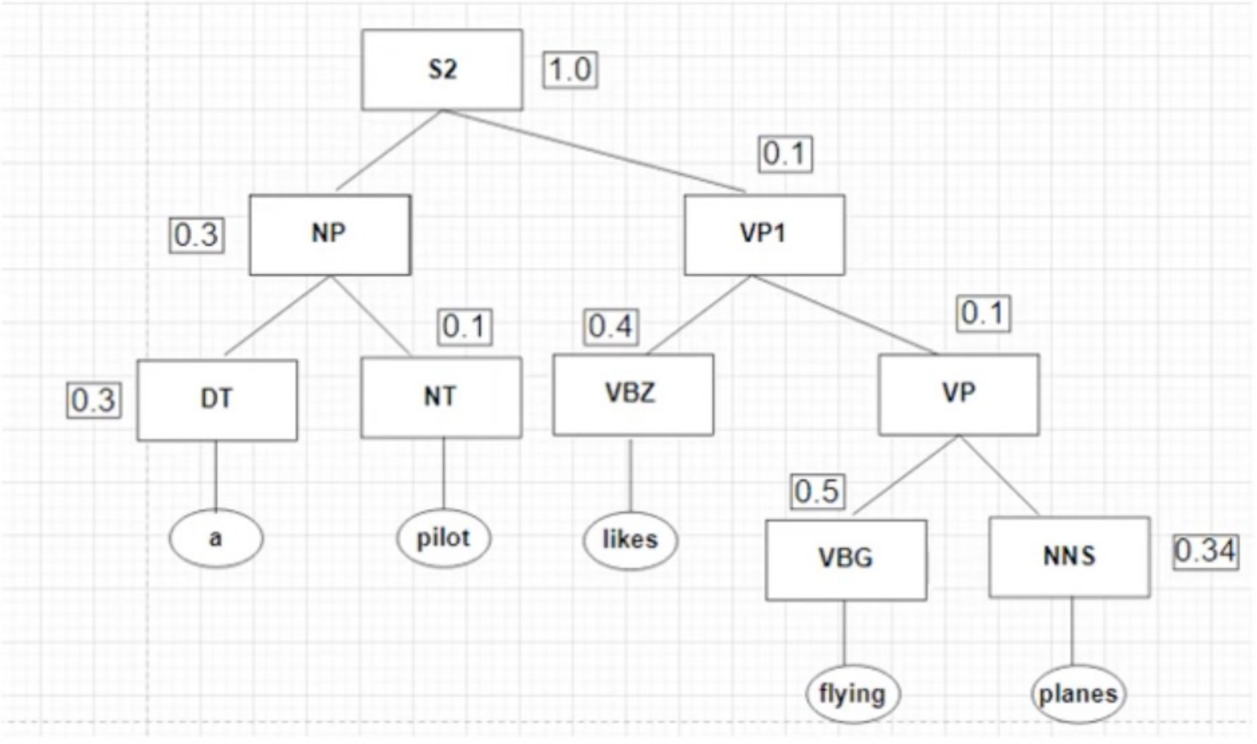


a	pilot	likes	flying	planes
DT (0.3) 01	NP (0.009) 02	03	04	S1 (0.000014688) S2 (0.00000612) 05
	NN (0.1) 12	--- 13	14	15
		VBZ(0.4) 23	--- 24	VP1 (0.001632) ,VP2 (0.00068) 25
			VBG (0.5), JJ (0.1) 34	NP (0.0136), VP (0.017) 35
				NNS (0.34) 45

VP1	NP	VBZ	
0.3	0.0136	0.4	0.001632
VP2	VP	VBZ	
0.1	0.017	0.4	0.00068

S1	NP	VP1	
1.0	0.009	0.001632	0.000014688
S2	NP	VP2	
1.0	0.009	0.00068	0.00000612





Rules	Probability
R={	
S -> NP VP	1.0
VP -> VBG NNS	0.1
VP -> VBZ VP	0.1
VP -> VBZ NP	0.3
NP -> DT NN	0.3
NP -> JJ NNS	0.4

Rules	Probability
DT-> a	0.3
NN->pilot	0.1
VBZ->likes	0.4
VBG->flying	0.5
JJ->flying	0.1
NNS->planes	0.34
}	

S1=0.3*0.1*0.4*0.4*0.1*0.34*0.3*0.3*1=0.000014688

S2=0.3*0.1*0.4*0.1*0.4*0.34*0.3*0.1*1=0.00000612

S1 is more probable

Advantages:

- **Disambiguation:** PCFG helps in disambiguating among multiple parse trees by choosing the one with the highest probability.
- **Statistical Basis:** Leverages statistical information from a corpus, providing a more realistic and data-driven approach to parsing.
- **Flexibility:** Can be adapted and trained on different corpora, making it versatile for various languages and domains.

Disadvantages

Data Dependency: Requires a large annotated corpus to accurately estimate probabilities, which might not be available for all languages.

Computational Complexity: Calculating probabilities and finding the most likely parse tree can be computationally intensive, especially for long sentences with many possible parse trees.

Sparsity: In cases where certain production rules are rare, probability estimates might be unreliable, leading to poorer performance.